Lab 4 – working with Apache Spark (own work in the laboratory)

Laboratory goal:

The aim of the laboratory is to learn how to transform data using the Apache Spark framework. Getting to know different formats for storing data in Apache Apark

A: Using the Internet, answer the following questions:

- what is ORC format?

- what is Parquet format

- what is Apache Spark and how it works with Parquet format and ORC fortmat?

Exercise1:

NOTE: We work on geolocation data and driver journeys (goods forwarding).

(Files: trucks.csv and geolocation.csv)

---

1. Analyze the data in the file: trucks.csv, geolocation.csv --> and consider:

- what information is stored in the file and how can this data be used for analysis?

- what will each column contain after importing into Hive?

- are the data of the file: trucks.csv matched to the processing using the information stored in the file: geolocation.csv?

- how many rows will be available in each table if You import this data into Hive?

- what theses (hypotheses) can be made when analyzing these data?

- how can the columns (and data) of this table be transformed so that this data can be processed?

- what languages, techniques and tools can be used to transform the data of these files (or tables) and determine how to achieve it?

Exercise2:

C. In the Apache Zeppelin environment, define a new project for Apache Spark preprocessing:

1. Analyze the data in the file: trucks.csv, geolocation.csv --> and consider:

- what information is stored in the file and how can this data be used for analysis?

- what will each column contain after importing into Apache Spark?

- are the data of the file: trucks.csv matched to the processing using the information stored in the file: geolocation.csv?

- how many rows will be available in each table if You import this data into Apache Spark?

- what theses (hypotheses) can be made when analyzing these data?

- how can the columns (and data) of this table be transformed so that this data can be processed?

2. Make an appropriate transformation of the data structure in the trucks.csv . To do this, write appropriate instructions.

2. Write example queries to geolocation data:

A.) - What is the total number of miles driven by each driver?

(or: how many miles a given driver has travelled since the beginning of trip recording) ?

B.) - What is the average fuel consumption for each truck? - think about whether it is possible to easily calculate the average fuel consumption with such stored data (the solution will be given during classes)?

3. Calculate the risk factor for each driver's road accident as the ratio of the number of accidents to the total number of miles traveled. Then save the data in tabular form in the riskfactor.csv file format.

Exercise3:

NYC Taxi Trip Data (Yellow Taxi) - Apache Spark Exercises:

https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

The New York City Taxi and Limousine Commission (NYC TLC) - Yellow Taxi dataset is one of the most popular and realistic datasets used for learning the Apache Spark ecosystem and data engineering. Each row in the file represents a single, completed trip in a famous New York yellow taxi, automatically recorded by the vehicles' onboard systems.

Business and Analytical Context

- Spatiotemporal dynamics: The dataset contains precise timestamps for the start and end of trips, as well as city zone identifiers (TLC Taxi Zones). This allows for mapping traffic intensity, identifying peak hours, and analyzing the flow of people between New York City boroughs.
- Financial structure: The dataset provides a detailed breakdown of the costs for each trip, isolating the base fare from state taxes, highway tolls, congestion surcharges, and tips. This enables the study of consumer behavior, including analyzing the correlation between the payment method and the propensity to leave a tip.

Data Quality and Challenges Working with this dataset perfectly simulates the problems analysts face in real-world Big Data projects. During data exploration and cleaning, you may encounter:

- Negative values: Trip charges (total_amount or fare_amount) falling below zero, usually resulting from refunds, disputes, or taximeter errors.
- Logical errors and anomalies: Trips with a distance of 0 miles but a charged fare; trips with a passenger count of 0; date errors where the trip end time is earlier than its start time.
- Missing data (Nulls): Empty values in columns manually entered by the driver or arising from a temporary loss of system connectivity.
- Hidden dependencies: Cash tips are not recorded by the system and appear as $0.00, which requires caution when drawing conclusions about the drivers' total earnings.

The Importance of the Parquet Format The file you will be working on is saved in the Apache Parquet format. Pay attention to how this format differs from classic CSV files:

- It stores data in a columnar format, which drastically speeds up analytical queries where you only use selected attributes (e.g., only distance and price).
- It has an embedded data schema (schema evolution and strong typing) – you do not need to manually cast text strings to numbers or dates after loading the file.
- It offers advanced dictionary compression, meaning it takes up a fraction of the disk space compared to raw text.

Basics and exploration (Equivalent to structure analysis from Lab.4):

1. Load the Parquet file with NYC Taxi data into a Spark DataFrame.

2. Display the schema of the loaded dataset and the first 20 rows.

3. Calculate the total number of records in the analyzed Parquet dataset.

4. Check if there are missing values (nulls) for key columns (e.g., passenger_count, trip_distance).

5. Calculate basic descriptive statistics (mean, minimum, maximum, standard deviation) for the total_amount and trip_distance columns.

Filtering and transformations (Equivalent to transformations from Lab.4 -ex.2):

6. Filter the dataset to keep only trips where the number of passengers was greater than 0.

7. Find trips with zero distance (trip_distance = 0) but an applied fare (fare_amount > 0) and count their occurrences.

8. Create a new column trip_duration_minutes by calculating the trip duration based on the start and end time columns.

9. Calculate the average speed of each trip (miles/hour) and save it in a new column average_speed.

10. Filter out anomalies: remove records where the total fare (total_amount) is negative.

Aggregations and analytical queries:

11. Group the data by payment type (payment_type) and calculate what percentage of all trips were paid by credit card.

12. Find the top 5 most popular pick-up zones (PULocationID) based on the number of started trips.

13. Calculate the average tip (tip_amount) for trips with 1, 2, 3, and 4+ passengers.

14. Extract the hour from the trip start time column and find the peak hour (the hour with the highest number of trips in a day).

15. Calculate the total revenue (total_amount) generated by each taxi operator (VendorID).

16. Check if there is a correlation between trip distance (trip_distance) and tip amount (tip_amount).

17. Calculate the ratio of the tip to the total fare (tip_percentage) for each trip paid by card.

Advanced operations and saving data:

18. Using Window Functions, find the longest trip for each day of the week.

19. Save the cleaned dataset (without anomalies, with the added duration column) back to a new file in Parquet format, partitioning it by the trip date.

20. Save the aggregated table with the daily revenue summary to a file in ORC format, to refer to the questions about formats from the Lab.4_exercises_EN.docx file.